
Rewiring Experts on the Fly: Continuous Rerouting for Better Online Adaptation in Mixture-of-Expert models

Guinan Su^{*1} Yanwu Yang^{*2} Li Shen³ Lu Yin⁴ Shiwei Liu¹ Jonas Geiping¹

Abstract

Mixture-of-Experts (MoE) models achieve efficient scaling through sparse expert activation, but often suffer from suboptimal routing decisions due to distribution shifts in deployment. While existing test-time adaptation methods could potentially address these issues, they primarily focus on dense models and require access to external data, limiting their practical applicability to MoE architectures. However, we find that, instead of relying on reference data, we can optimize MoE expert selection on-the-fly based only on input context. As such, we propose a *data-free, online test-time framework* that continuously adapts MoE routing decisions during text generation without external supervision or data. Our method cycles between two phases: During the prefill stage, and later in regular intervals, we optimize the routing decisions of the model using self-supervision based on the already generated sequence. Then, we generate text as normal, maintaining the modified router until the next adaption. We implement this through lightweight additive vectors that only update router logits in selected layers, maintaining computational efficiency while preventing over-adaptation. The results show consistent performance gains on challenging reasoning tasks while maintaining robustness to context shifts. For example, our method achieves a 5.5% improvement on HumanEval with OLMoE. Furthermore, owing to its plug-and-play property, our method complements existing test-time scaling techniques, e.g., achieving 6% average gains when incorporated with self-consistency on DeepSeek-V2-Lite.

1. Introduction

Mixture-of-Experts (MoE) models (Shazeer et al., 2017; Zhou et al., 2022; Jiang et al., 2024; Dai et al., 2024; Liu et al., 2024; Team, 2025; Muennighoff et al., 2024) provide an effective approach to scaling model capacity while maintaining computational efficiency by partitioning parameters into specialized experts and selectively activating subsets through routing mechanisms (Lepikhin et al., 2020; Fedus et al., 2022; Dai et al., 2024; Muennighoff et al., 2024). This functionality enables dynamic expert selection for diverse queries and creating inherently general-purpose systems that can store much more functionality and information than is used in every forward pass. However, despite their impressive capabilities, MoE models still face challenges when deployed in real-world environments (Lyu et al., 2026; Li et al., 2025c), as the expression of their capability hinges on the quality of their *routing decisions*, the activations of small linear layers that determine which parts of the model are activated.

Why is routing hard? While the full MoE may store sufficient functionality to solve a particularly challenging query, this capacity is gated behind its routing decisions in each MoE layer, which in turn depend on the residual stream of the model, and so on earlier routing decisions. Routing decisions are only linear functions of the current hidden state that need to approximate the anticipated utility of activating a certain expert. A particular issue with this non-robustness of routing is that during standard inference *there is no mechanism to reinforce the routing to a particularly successful part of the model*, or to reduce routing to parts that did not contribute meaningful signal to the generated text.

This makes routing optimization a question of model adaptation, reminiscent of neuroplasticity in humans – who continuously optimize routing and neuronal connections in the brain through adaptation and self-regulation, for example when continuing to practice a certain task. Yet, in MoEs, suboptimal expert selection leads to routing inefficiencies, creating a critical bottleneck in overall performance (Shi et al., 2024; Li et al., 2025c). And, while there has been a significant amount of work to improve the capabilities of MoEs in general, such as through test-time scaling, it is less clear how to best adapt MoEs to different tasks at inference.

^{*}Equal contribution ¹Max Planck Institute for Intelligent Systems & ELLIS Institute Tübingen & Tübingen AI Center ²University of Tübingen ³Shenzhen Campus of Sun Yat-sen University ⁴University of Surrey. Correspondence to: Guinan Su, Yanwu Yang, Li Shen, Jonas Geiping <guinansu33@gmail.com>.

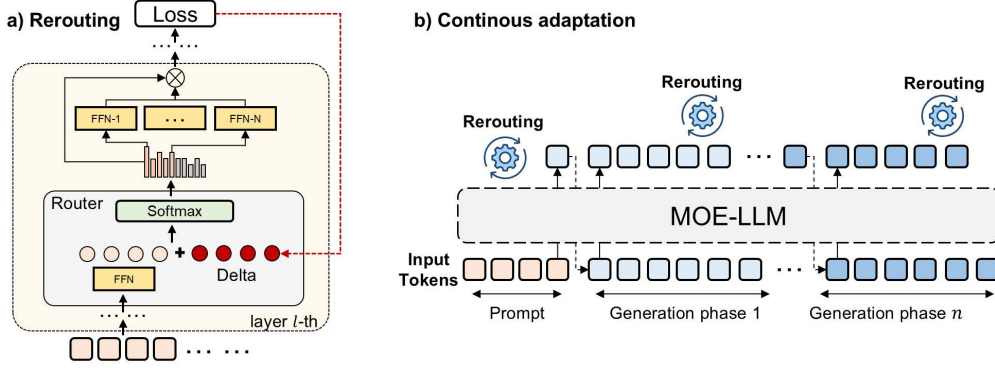


Figure 1. Test-time rerouting framework for MoE models. (a) **Rerouting mechanism:** lightweight additive vectors (Delta) update router logits in selected layers using self-supervised loss from existing context. (b) **Continuous adaptation:** alternating between optimization phases that adapt routing decisions and generation phases that maintain adapted routing until the next optimization cycle.

While standard approaches, such as in-context learning with task-specific demonstrations (Wei et al., 2022; Madaan et al., 2023) or parallel generation strategies that produce multiple candidates and aggregate them (Wang et al., 2022; Brown et al., 2024), do adapt the model overall, they do influence routing only implicitly.

Conceptually, adaptation should be solvable by *test-time training*, but existing approaches (Hardt & Sun; Hübottner et al., 2024) focus on a classical prediction perspective, which retrieves relevant data from training sets or knowledge bases during inference to fine-tune models for dynamic scenarios before the model is being used. Li et al. (2025c) attempt to address router optimization through this perspective of test-time training, and, as such, use retrieval of "successful neighbors" from reference sets based on the first prompt in each context. However, this approach requires access to external reference data during deployment, incurs retrieval overhead, and risks failures in retrieval due to short prompts. Moreover the approach is static, as modern models execute test-time scaling, they generate long chains-of-thought, that should be taken into account when optimizing routing.

In this regard, we propose a simple yet effective *data-free* test-time rerouting framework for MoE models as shown in Figure 1 that treats each input prompt as a self-supervised learning opportunity, enabling dynamic rerouting of pre-trained MoE models for individual prompts during inference. The framework alternates in two phases: (1) **In-Context Routing Optimization**, where we regard the current context itself as a training sample and execute optimization steps to minimize cross-entropy loss on the current context with regard to routing logits; and (2) **Steered Generation**, where we generate text normally, steering routers with updates computed in the previous phase. This creates a dynamic feedback loop where *the model continuously refines its understanding of task requirements* based on its own generation progress, enabling increasingly informed expert routing decisions as generation proceeds. To maintain computa-

tional efficiency, we implement this progressive optimization through **lightweight additive parameter vectors that update only the model’s router logits of selected MoE layers**, further reducing computational overhead and preventing over-adaptation. Overall, our main contributions are:

- We propose a test-time rerouting framework specifically for MoE models, operating without external data dependencies or expensive retrieval mechanisms, using only backpropagation within the current context.
- We introduce a lightweight parameter update mechanism that selectively optimizes router logits through additive vectors, enabling efficient expert selection adaptation during inference using steering.
- We validate our method’s effectiveness through various scenarios, demonstrating that our approach significantly improves MoE model performance on complex reasoning tasks while maintaining computational efficiency and generation diversity.

Overall, the results demonstrate that our routing optimization provides an effective mechanism for data-free test-time adaptation of MoE models. The changes are lightweight and fast to apply, and endow MoEs with a new degree of plasticity for adapting to task shifts during deployment, enabling practical continual self-regulation at inference time.

2. Related Work

Mixture-Of-Experts (MoE). Sparsely activated Mixture-of-Expert models (MoE) are an efficient method for scaling model size by replacing the feed-forward networks (FFN) of transformer architectures with multiple experts (each its own FFN) and a gating function. This architecture dynamically activates different experts for each input token rather than utilizing all parameters (Shazeer et al., 2017; Lepikhin et al., 2020; Fedus et al., 2022). Recent MoE-based LLMs, including OLMoE (Muennighoff et al., 2024), DeepSeek (Dai et al., 2024; Liu et al., 2024), and Qwen2.5 (Team, 2025),

adopt top-k expert routing to reduce the active parameter count during inference. These models achieve competitive performance while maintaining lower memory costs.

Test-Time Scaling. Test-Time Scaling enhances LLM capabilities by allocating additional computational resources during inference (Li et al., 2026; 2025a). These approaches fall into two categories (Welleck et al., 2024): Parallel generation, including self-consistency evaluations via multiple candidate responses (Wang et al., 2022), best-of-N sampling (Brown et al., 2024), and Monte Carlo Tree Search (Zhou et al., 2023; Xie et al., 2024), and sequential generation such as extending outputs through chain-of-thought reasoning (Wei et al., 2022; Madaan et al., 2023).

Test-Time Training (TTT). TTT offers an alternative scaling approach. While successful in computer vision (Wang et al., 2020; Sun et al., 2020; 2024; Gandelsman et al., 2022; Osowiechi et al., 2023), recent works have extended TTT to language models through fine-tuning on retrieved neighbors (Hardt & Sun) or optimized data selection algorithms like SIFT (Hübötter et al., 2024). Test-Time Reinforcement Learning (TTRL) (Zuo et al., 2025) uses majority voting as reward signals. When applied to Mixture-of-Experts (MoE) models, recent work has explored expert-level interventions for behavior control, enabling precise modifications through targeted expert manipulation (Dahlke et al., 2025; Wang et al., 2025). Most relevant to our approach, Li et al. (2025c) optimizes expert routing in MoE models using "successful neighbors" from reference sets. However, these methods assume accessible training data during deployment and introduce retrieval overhead, limiting real-world practicality.

3. Methodology

This section presents our data-free test-time rerouting framework for MoE models that optimizes expert routing during inference without external information. After reviewing MoE fundamentals (Section 3.1), we introduce three key components: (1) Router Logits Modification (Section 3.2) for layer-specific expert selection steering, (2) Dynamic Layer Selection (Section 3.3) for selective MoE layer updates based on specialization scores, and (3) Optimization Procedure (Section 3.4) detailing our two-phase strategy alternating between in-context routing optimization and steered generation.

3.1. Preliminaries on Mixture-of-Experts

In Transformer-based Mixture-of-Experts (MoE) models, the conventional Feed-Forward Networks (FFNs) are replaced with MoE layers. Each MoE layer consists of a router R and a set of experts $\{E_i\}_{i=1}^N$. Given an input sequence $\mathbf{x}_{<t} = (x_1, x_2, \dots, x_{t-1})$, the router assigns each token to a subset of experts for processing. Given

a token’s hidden state $\mathbf{h} \in \mathbb{R}^d$ at layer l , the router computes logits across all N experts: $\mathbf{z}^{(l)} = W_r^{(l)} \mathbf{h}^{(l)}$ where $W_r^{(l)} \in \mathbb{R}^{N \times d}$ is the router’s weight matrix at layer l , and $\mathbf{z}^{(l)} \in \mathbb{R}^N$ represents the logits for expert selection. These logits are then converted to expert selection probabilities: $\mathbf{w}^{(l)} = \text{Softmax}(\mathbf{z}^{(l)})$ where $w_i^{(l)}$ represents the activation probability for expert E_i at layer l . The router applies a routing strategy (e.g., top- k routing) to select active experts. Weights for unselected experts are zeroed and the remaining weights are renormalized to $\tilde{\mathbf{w}}^{(l)}$. The final MoE output is: $\mathbf{o}^{(l)} = \sum_{i \in \mathcal{A}^{(l)}} \tilde{w}_i^{(l)} \cdot E_i(\mathbf{h}^{(l)})$ where $\mathcal{A}^{(l)} = \{j \mid \tilde{w}_j^{(l)} \neq 0\}$ denotes the set of activated experts at layer l .

3.2. Router Logits Modification

We introduce layer-specific adaptation parameters $\{\delta^{(l)}\}_{l=1}^L$ where $\delta^{(l)} \in \mathbb{R}^N$ corresponds to the N experts at MoE layer l . For a selected layer l , we modify the router logits by adding the corresponding layer-specific parameter: $\tilde{\mathbf{z}}^{(l)} = \mathbf{z}^{(l)} + \delta^{(l)}$ where $\tilde{\mathbf{z}}^{(l)} \in \mathbb{R}^N$ represents the modified logits for layer l . The expert selection probabilities are then computed as:

$$\tilde{\mathbf{w}}^{(l)} = \text{Softmax}(\tilde{\mathbf{z}}^{(l)}) = \text{Softmax}(\mathbf{z}^{(l)} + \delta^{(l)}) \quad (1)$$

This modification directly influences the expert selection distribution, allowing the model to adapt routing decisions based on prompt characteristics.

3.3. Dynamic Layer Selection

To mitigate computational overhead and prevent over-adaptation, our framework selectively updates router logits of only a subset of MoE layers rather than all layers simultaneously. We hypothesize that layers with higher specialization values are more impactful for adaptation, as their routing decisions play a more decisive role in expert selection. We define expert specialization score at layer i for token m as:

$$C_i^{(m)} = -\frac{1}{k} \sum_{j=1}^k \log p_{i,j}^{(m)} \quad (2)$$

where $p_{i,j}^{(n)}$ is the probability of the j -th top expert at layer i for token m , and k is the number of activated experts per layer. *Higher values indicate more concentrated and confident expert routing within the layer, suggesting that these layers play a more prominent role in task-specific test-time adaptation.* Note that this is *not* Shannon entropy. See Appendix B.3 for a detailed comparison.

To obtain layer-level specialization score across the generated sequence, we aggregate token-level specialization score:

$$C_i = \frac{1}{M} \sum_{m=1}^M C_i^{(m)} \quad (3)$$

where M is the number of generated tokens so far. We implement two layer selection strategies: (1) **Hard selection** that selects top- r proportion of layers with highest specialization scores: $\mathcal{S}_t = \text{TopK}(\{C_i\}_{i=1}^L, r)$, and (2) **Soft weighting** that assigns specialization score-based weights to control the update strength of $\delta^{(l)}$:

$$w_i = \frac{C_i}{\sum_{j=1}^L C_j} \quad (4)$$

During gradient updates, $\delta^{(l)}$ is updated with learning rate scaled by w_l .

3.4. Optimization Procedure

Parameter Initialization: For each MoE layer $l \in \mathcal{L}$, initialize routing adjustment parameters:

$$\delta^{(l)} = \mathbf{0} \in \mathbb{R}^N \quad (5)$$

The framework alternates between two phases:

Phase 1: In-Context Routing Optimization. Given current context $\mathbf{x} = (x_1, \dots, x_t)$, first select layers using $\mathcal{S} = \text{TopK}(\{C^{(l)}\}_{l=1}^L, r)$ or compute soft weights $\{w_l\}_{l=1}^L$. Then perform n optimization steps, where at each step we compute the cross-entropy loss:

$$\mathcal{L}(\{\delta^{(l)}\}_{l=1}^L) = - \sum_{i=1}^{t-1} \log p(x_{i+1} \mid x_{1:i}, \{\delta^{(l)}\}_{l=1}^L) \quad (6)$$

and update parameters using optimizer $\mathcal{O}$ (e.g., SGD, Adam):

$$\delta^{(l)} \leftarrow \begin{cases} \mathcal{O}(\delta^{(l)}, \nabla_{\delta^{(l)}} \mathcal{L}) & \text{if } l \in \mathcal{S} \text{ (hard selection)} \\ \mathcal{O}(\delta^{(l)}, w_l \nabla_{\delta^{(l)}} \mathcal{L}) & \text{if soft weighting} \\ \delta^{(l)} & \text{otherwise} \end{cases} \quad (7)$$

Phase 2: Steered Generation. Generate m tokens using optimized routing parameters $\{\delta^{(l)}\}_{l=1}^L$.

After generating m tokens, return to Phase 1 with extended context including both original prompt and all generated tokens up to the current position and repeat the optimization procedure. More details are provided in [Algorithm 1](#).

4. Experiments

Benchmarks. We evaluate our approach using a diverse set of benchmarks across multiple reasoning domains. For general knowledge assessment, we employ MMLU-redux ([Gema et al., 2024](#)), utilizing generation mode rather than multiple-choice format to encourage deeper reasoning before providing final answers. For code generation tasks, we use HumanEval ([Chen et al., 2021](#)) and MBPP-sanitized

([Austin et al., 2021](#)). For mathematical reasoning, we evaluate on GSM8K ([Cobbe et al., 2021](#)) and MATH500 ([Lightman et al., 2023](#)).

Baselines. We compare our method with two adaptation techniques: In-Context Learning (ICL) ([Wei et al., 2022](#); [Madaan et al., 2023](#)) and C3PO ([Li et al., 2025c](#)). Since our method is a data-free sequential generation test-scaling approach, we select ICL as a primary baseline, using 3 and 5 sample pairs respectively, for comparison. We also compare against C3PO, for which we select a reference set of 100 samples for each dataset. Note that parallel generation methods represent an orthogonal approach to ours, and we further discuss the potential integration of our method with such approaches in later sections.

Model Selection. We evaluate three MoE LLMs: OLMoE ([Muennighoff et al., 2024](#)), Qwen1.5-MoE ([Team, 2024](#)), and DeepSeek-V2-Lite ([Liu et al., 2024](#)). OLMoE employs 16 layers with 64 experts per layer, activating 8 experts per token (6.9B total, 1.3B active parameters). Qwen1.5-MoE-A2.7B, incorporates 4 shared experts alongside 60 routing experts with 4 activated per token (14.3B total, 2.7B active parameters). DeepSeek-V2-Lite uses 28 layers with 2 shared and 64 routed experts, activating all shared plus 6 routed experts per token (16B total, 2.4B active parameters).

Optimization. We optimize the adaptation parameters using the Adam optimizer with a small number of iterations ($T = 5$) to maintain computational efficiency. The optimization hyperparameters are set as follows: learning rate $\eta = 0.05$, weight decay $= 1 \times 10^{-8}$, and epsilon $= 1 \times 10^{-5}$. All adaptation parameters are initialized to zero: $\delta^{(l)} = \mathbf{0}$. We use the soft-weighting strategy for layer selection, and during the generation stage, we periodically re-optimize the parameters during generation using identical hyperparameter settings to ensure continuous refinement throughout the sequence generation process. Specifically, we set the re-optimization interval to 128 tokens for math and MMLU tasks, and 96 tokens for code generation tasks.

5. Results

5.1. Main results

We first evaluated the performance of our method on challenging reasoning benchmarks. As shown in [Table 1](#), our approach consistently improves performance over all baselines across five benchmarks. On the HumanEval task in particular, our method yields gains of 3.6%, 5.5%, and 6.7% on DeepSeek-V2-Lite, OLMoE, and Qwen1.5-MoE, respectively. Moreover, compared to other calibration methods such as few-shot learning and C3PO, our method achieves consistently better results. Notably, it surpasses these baselines without relying on example demonstrations, delivering particularly strong improvements in code generation and

mathematical reasoning tasks. Moreover, in contrast to C3PO, which requires 100 reference samples, our data-free method attains superior performance while requiring no additional reference data.

Table 1. Model Performance Comparison Across Different Benchmarks

Method	HumanEval	MBPP	GSM8K	MATH500	MMLU	Average
DeepSeek-V2-Lite						
Baseline	50.60	58.37	72.10	22.60	50.77	50.89
ICL (3-shot)	49.85	56.57	72.32	22.50	44.33	49.11
ICL (5-shot)	50.61	56.50	72.84	22.80	46.07	49.76
C3PO (100-reference)	47.80	59.92	68.80	18.20	45.77	48.10
Ours (0-shot)	54.26	62.65	73.62	25.00	52.40	53.59
OLMoE						
Baseline	28.66	40.08	51.48	10.40	37.17	33.56
ICL (3-shot)	28.05	36.96	43.82	9.90	44.90	32.73
ICL (5-shot)	21.21	39.30	44.96	9.40	43.63	31.70
C3PO (100-reference)	28.05	41.25	52.08	13.00	37.30	34.33
Ours (0-shot)	34.17	42.80	52.99	12.40	38.30	36.13
Qwen1.5-MoE						
Baseline	40.24	46.69	54.73	21.80	45.27	41.75
ICL (3-shot)	44.34	46.25	55.27	22.20	45.60	42.73
ICL (5-shot)	43.90	44.75	52.31	19.40	46.03	41.28
C3PO (100-reference)	39.70	45.40	50.33	18.90	44.82	39.83
Ours (0-shot)	46.95	47.47	56.00	22.00	45.87	43.66

Table 2. Ablation Study: Layer Selection Strategies and Continuous Refinement

Method	HumanEval	MBPP	GSM8K	MATH500	MMLU	Average
Baseline	50.60	58.37	72.10	22.60	50.77	50.89
<i>Layer Selection Strategies</i>						
Random Selection	48.78	53.31	72.50	23.40	50.87	49.77
Expert-diffuse Layers	48.18	55.26	72.10	21.89	49.33	49.35
Last-five Layers	50.60	55.64	71.85	25.20	50.33	50.72
All Layers	48.17	56.42	73.09	24.60	51.10	50.68
<i>Continuous Refinement</i>						
W/o Continuous	54.27	59.53	73.37	23.80	51.80	52.55
Ours	54.27	62.65	73.62	25.00	52.40	53.59

5.2. Ablation and Sensitivity Analysis

To validate the effectiveness of our design choices, we conduct ablation studies on using DeepSeek-V2-Lite, examining two key components: layer selection strategies and continuous refinement mechanisms. We evaluate performance across five tasks, with results presented in Table 2.

Layer specialization improves test-time gating refinement. To evaluate the effectiveness of layer selection, we compare our expert-specialized layer strategy against several baselines: random layer selection, selecting the last five layers (as adopted in C3PO), selecting expert-diffuse layers (in opposite to our expert-specialized layers), and optimizing all layers. As shown in Table 2, our method achieves the best average performance of 53.59% across tasks, outperforming random selection (49.77%), the last-five-layer strategy (50.72%), expert-diffuse layer selection (49.35%), and all-layer optimization (50.68%). Notably, selecting expert-diffuse layers leads to consistent performance degradation across all benchmarks, indicating that uncertain layers are poorly suited for test-time optimization.

Selected-layer δ refinement outperforms all-layer refinement. In addition, from the results we can see that updating all layers simultaneously (50.68% on average) underperforms our selective approach (53.59% on average), highlighting the superiority of our method in selecting layers for test-time rerouting. With limited optimization steps available, spreading updates across all parameters dilutes the refinement signal and risks overadaptation that destabilizes pre-trained routing patterns. Concentrating the optimization budget on high specialization score layers maximizes impact by amplifying existing routing strengths rather than attempting comprehensive corrections across the entire network.

Continuous refinement stabilizes routing. The comparison between continuous (53.59% on average) and non-continuous refinement (52.55% on average) reveals a meaningful performance gap of over 1%. It is notable that this gap carries weight, given that each benchmark example involves a single task and prompt-based router updates already constitute a strong baseline. This improvement indicates that routing optimization benefits from curriculum-like approaches that gradually refine the model’s expert selection mechanisms, leading to more stable and effective adaptations during test-time optimization. See more results in Section C.

5.3. Combining rewiring with other test-time strategies

A key advantage of our approach is its plug-and-play compatibility with existing test-time techniques. Since our method only modifies routing decisions without altering the underlying generation process, it can be seamlessly integrated with other approaches like in-context learning and parallel generation methods.

Synergy with In-Context Learning. Large pretrained language models demonstrate strong in-context learning, predicting labels from few demonstration pairs without parameter updates. Empirical evidence demonstrates behavior similar to explicit finetuning (Dai et al., 2022; Akyürek et al., 2022; Von Oswald et al., 2023). As such, we test combining our method with 3-shot demonstrations. As shown in Table 3, we observe that this combination yields improvements across multiple tasks, even surpassing our method alone. We hypothesize this synergy occurs because our method provides more effective gradient updates that better leverage the contextual information from demonstration examples, helping extract more meaningful patterns from the limited demonstration data.

Unlocking better self-consistency. Self-consistency (Wang et al., 2022) improves reasoning by sampling multiple paths and aggregates together. We combine our method with self-consistency by applying rerouting optimization, then generating multiple reasoning paths with optimized routing decisions. For each sample, we generate 3 reasoning paths. For MMLU, MATH500, and GSM8K, we use majority voting

to determine the final accuracy. For code generation tasks (MBPP, HumanEval), we report pass@3 scores. The results in Table 3 show substantial improvements with an average 3 percentage point gain over self-consistency alone. We hypothesize that our rerouting framework generates higher-quality reasoning chains by selecting more appropriate experts, and when self-consistency aggregates these improved paths, the voting mechanism amplifies the benefits.

Table 3. Performance comparison of individual Test-Time methods versus combined approaches.

Method	HumanEval	MBPP	GSM8K	MATH500	MMLU	Average
Baseline	50.60	58.37	72.10	22.60	50.77	50.89
Ours	54.26	62.65	73.62	25.00	52.40	53.59
<i>In-Context Learning</i>						
ICL (3-shot)	52.44	56.81	71.10	21.60	44.33	49.26
ICL + Ours	53.05	62.65	76.00	27.00	46.53	53.05
<i>Self-Consistency</i>						
Self-Consistency (3)	51.02	70.04	75.28	26.20	51.87	54.88
Self-Consistency (3) + Ours	55.08	71.21	77.54	27.40	54.20	57.09

Table 4. Performance of our method versus the baseline on AIME datasets using the *GPT-OSS-20b* model. Optimizing rerouting is especially noticeable at improving model certainty, as shown by improved performance at lower pass@k and improved majority voting, even on one challenging benchmarks like AIME.

Data / Method	Pass@k			Maj@k			Average
	2	4	8	2	4	8	
<i>AIME24</i>							
Baseline	83.21	87.90	90.00	65.36	71.76	76.67	74.29
Rewiring (Ours)	83.81	86.19	86.67	67.50	76.67	83.33	75.65
<i>AIME25</i>							
Baseline	78.57	83.76	86.67	60.60	67.48	70.00	69.58
Rewiring (Ours)	81.67	84.95	86.67	65.83	72.57	80.00	73.75

5.4. Extension to Long-Reasoning Tasks

To evaluate the generalizability of our approach to modern reasoning models, we also test GPT-OSS-20B (Agarwal et al., 2025) on the AIME benchmark (MAA Committees), a challenging mathematics competition dataset that requires sophisticated multi-step derivations and mathematical reasoning. This setting allows us to assess whether our online rerouting method can improve performance on the demanding reasoning tasks that represent the current frontier of AI capabilities. As shown in Table 4, our method improves performance on both AIME datasets, achieving higher average correctness (75.65% vs. 74.29% on AIME24; 73.75% vs. 69.58% on AIME25). The primary improvements occur in Maj@k metrics rather than Pass@k, suggesting our routing optimization enhances reasoning consistency rather than solution diversity.

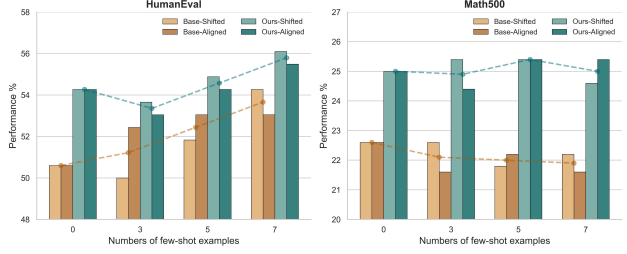


Figure 2. Performance of our method versus the baseline across different few-shot examples under shifted and aligned contexts.

5.5. Robustness to Multi-turn Scenarios.

In real-world applications, MoE models often encounter *multi-turn conversations* where contexts shift dramatically between different topics or tasks. To assess the robustness of our method under realistic multi-turn contexts, we simulate context shifts by prepending few-shot examples from different domains before the target task. Figure 2 presents results on HumanEval and Math500 using DeepSeek-V2-Lite, under two conditions: (1) **Aligned-task**, where few-shot examples are from the same task domain, and (2) **Shifted-task**, where examples are drawn from unrelated domains (e.g., MATH and MMLU for code tasks) to induce *cross-domain shifts*.

Across both benchmarks, our method (Ours-Shifted, Ours-Aligned) consistently outperforms the baselines (Base-Shifted, Base-Aligned). On HumanEval, all methods benefit from more-shot examples, but our approach shows a stable improvement. In contrast, baseline gains remain modest, particularly in the shifted-task setting, where performance fluctuates. On Math500, our method maintains robust and consistent results across both domains, while baselines exhibit limited or even inconsistent improvements. These findings highlight the robustness and scalability of our test-time rerouting strategy in handling diverse contextual information.

5.6. Efficiency Analysis

Beyond performance improvements, we also compare the computational cost of our method with other online approaches to adapt models. Table 5 reports the results on the HumanEval task with the DeepSeek model, showing the average total FLOPs and inference time per sample across different methods. As shown in Table 5, our method achieves notable computational savings over most baselines. Although it requires more computation than the vanilla baseline, it remains more efficient than other test-time techniques, using 1.3× fewer FLOPs than C3PO, 1.6× fewer than ICL (5-shot), and comparable to ICL (3-shot) and Self-Consistency (3). Although ICL-based methods achieve shorter inference times, they often exhibit unstable and oc-

casionally degraded performance, as is shown in Table 1. In contrast, other regularization-based approaches, such as self-consistency and C3PO, still fall behind our method in terms of FLOPs, inference time, and overall performance gains.

Conceptually, our method requires n additional prefill passes on already generated text every m tokens, which, given the ease of parallelization of prefill is a manageable compute increase. In this work, we implement this optimization in a straightforward manner, but a production implementation could be significantly faster by incorporating the optimization into disaggregated-prefill systems, or timing the MoE rewiring event with low-load timespans, for example when waiting for a user to respond. The actual routing changes are self-contained in routing parameters δ (shaped as number of experts by number of layers), so that on-device storage is feasible, even for many separate conversations.

Table 5. Computational efficiency comparison across different methods.

Method	Total FLOPs	Time (s)
Baseline	4.71e+11	10.71s
ICL (3-shot)	1.93e+12	12.17s
ICL (5-shot)	3.05e+12	12.53s
Self-Consistency (3)	1.68e+12	34.20s
C3PO (100-reference)	2.63e+12	26.20s
Ours	1.96e+12	20.12s

6. How Does Test-Time Rerouting Work?

To better understand the mechanisms underlying test-time rerouting, we conduct a comprehensive analysis of examining pathway shifts, expert utilization, and the evolution of routing confidence on the DeepSeek-V2-Lite model.

Rerouting Induces Selective Reshaping of Expert Pathways. We first examine how expert pathways change after rerouting using edit distance following (Li et al., 2025b) (described in Section B.2), which quantifies pathway differences using Levenshtein edit distance. This captures mismatches in expert selection, and pathway shifts. We track the average pairwise edit distance across samples during rerouting. The results are displayed in Figure 3 a), which shows distinct layer-wise editing patterns. In HumanEval, edit distances peak in deeper layers (18–22, >0.35), while earlier ones (5–10) remain minimal (<0.05). MATH500 exhibits a similar but stronger trend, with peaks up to 0.16 in layers 20–22, suggesting that deeper layers are more involved in mathematical reasoning. The sample-wise distributions (right panels) further emphasize task-specific differences, indicating high variability in pathway changes across problems. These results highlight the heterogeneity of expert routing across tasks and demonstrate how our method adaptively reroute pathways.

Rerouting Induces Task-dependent Shifts in Expert Activation. We further examine expert-level activation shifts on HumanEval and MATH500 before and during rerouting to understand how the expert routings are redistributed. Figure 3b) shows heatmaps across layers and experts (red: increased, blue: decreased; normalized to $[-1, 1]$ for visualization). The results reveal three key patterns: (1) Selective adaptation: activation changes concentrate on a subset of experts, indicating structured rather than uniform redistribution; (2) Deep-layer modulation: later layers (L21–L25) undergo stronger shifts, consistent with our edit-distance analysis; and (3) Task-dependent behavior: HumanEval and MATH500 exhibit distinct activation patterns, reflecting task-specific routing dynamics. These observations characterize how rerouting redistributes computation across experts in a selective and task-dependent manner.

Rerouting Induces Increased Routing Confidence. Moreover, we quantify routing confidence by tracking the entropy of expert-selection distributions across all layers during optimization. Lower entropy reflects more focused expert allocation, whereas higher entropy indicates more diffuse routing patterns. Figure 4 shows that our method (red line) exhibits a gradual decrease in entropy over 18 generation steps, whereas the baseline (blue line) maintains higher entropy with pronounced fluctuations. This trend indicates that our approach progressively develops more focused routing decisions, while the baseline continues to rely on more diffuse expert-selection patterns. Overall, these results suggest that rerouting increases routing confidence by concentrating computation on task-relevant experts without disrupting the established routing structure. Moreover, this consistency suggests that test-time adaptation evolves in a coherent and self-reinforcing manner, progressively refining expert routing decisions without external supervision.

Summary. Overall, our findings demonstrate that rerouting facilitates structured test-time adaptation in MoE models by selectively editing expert pathways rather than applying global perturbations. This process redistributes expert activations in a task-dependent manner and progressively concentrates expert selection, thereby increasing routing confidence during generation. These architectural shifts are most prominent in deeper, task-relevant layers, whereas early layers remain stable, indicating that rerouting induces targeted, coherent adjustments rather than arbitrary noise. Mechanistically, although rerouting optimizes decisions based on previously generated text, it enhances future predictions by refining the internal representations of the context. Based on these observations, we suggest that these improvements may stem from two potential mechanisms: (1) Representation Refinement: High-quality intermediate representations, driven by improved reconstruction, provide a superior foundation for subsequent generation. (2) Expert Specialization Alignment: By minimizing reconstruction loss, the model

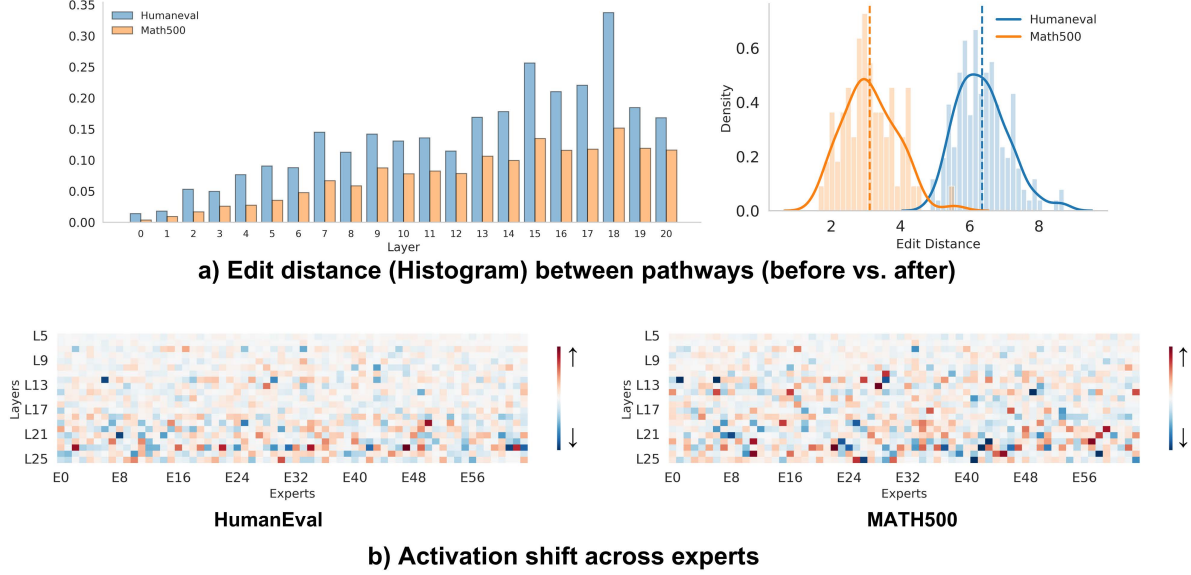


Figure 3. Analysis of test-time rerouting mechanisms across different datasets in DeepSeek-V2-Lite. a) Edit distance before and after rerouting. b) Expert utilization dynamics before and after rerouting.

better identifies and activates experts tailored to specific data subspaces. This is particularly effective under distribution shifts, where pre-trained routing often defaults to suboptimal experts for out-of-distribution test data.

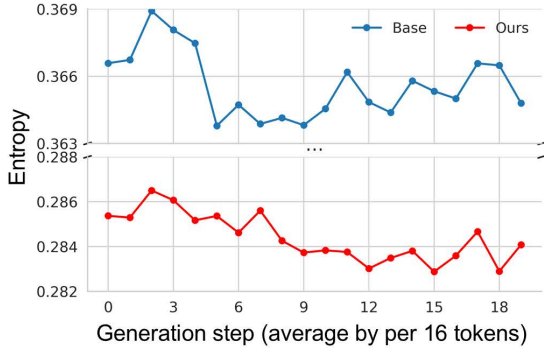


Figure 4. Expert routing entropy evolution during generation.

7. Discussion and Conclusion

Limitations and Future Work. First, our approach optimizes routing decisions using already-generated text, relying on the premise that refining how the model processes the observed context can improve related task-specific predictions. This is a general limitation of such source-free test-time adaptation methods, where updates depend on unlabeled target data and model self-predictions and may suffer from mismatch. Despite these limitations, recent works show that carefully constrained test-time updates can improve robustness and adaptability, and that this principle extends to MoE models that can be adapted online with-

out external supervision (Sun et al., 2020; Li et al., 2025d). These studies supports our assumption that leveraging internal structure can be effective in practice. A second concern is that excessive adaptation may overfit to a specific context or amplify early mistakes, leading to error accumulation or confirmation bias. Nevertheless, our method achieves consistent improvements across benchmarks and shows relatively robust performance under distribution shifts (Section 5.5). We mitigate these risks by limiting the number of optimization steps and selectively updating only a subset of layers. Future work could explore richer optimization objectives beyond our current emphasis on important neurons for sample-specific adaptation, for example, integrating real-time reward signals during generation to enable dynamic correction and adjustment, opening new directions for test-time optimization.

Conclusion. In this work, we introduce a simple yet insightful test-time rerouting approach that enables MoE models to dynamically adapt expert selection without requiring external data or costly retrieval. The method alternates between routing optimization and steered generation, forming a feedback loop that progressively improves expert selection. To reduce computational overhead, we employ lightweight additive vectors that update only the logits of selected routers. Extensive experiments show that our approach effectively compensates for the inherent imperfections in MoE routing, yielding consistent gains across multiple benchmarks (up to 6.7% on code generation) with 1.6× fewer FLOPs than few-shot methods, while maintaining robustness to context shifts. As a plug-and-play regularization strategy, the method flexibly combines

with complementary techniques (e.g., Self-Consistency) to further amplify the benefits. More importantly, by introducing a new dimension of plasticity into MoEs, it opens the door to deployment-time adaptation and points toward practical continual self-regulation in MoE models.

Impact Statement

Mixture-of-Experts models are increasingly deployed in real-world systems due to their scalability and efficiency, yet their routing remains static at inference time after deployment, often limiting their ability to adapt expert selection when encountering out-of-distribution scenarios. This work introduces a lightweight, plug-and-play rerouting mechanism that enables MoE models to adapt expert selection dynamically at test time, without relying on external data or retrieval. By demonstrating that such adaptations can be achieved efficiently and robustly across complex reasoning tasks and multi-turn interactions, our approach suggests a practical path toward more flexible and responsive deployment of large models. The ability to introduce controlled plasticity at inference time may reduce the need for frequent retraining or heavy prompting strategies, potentially lowering computational costs and energy consumption in deployed systems. Our analysis and ablations aim to shed light on when and why rerouting is beneficial, providing tools for more principled use of test-time adaptation. More broadly, this work points toward a future in which large-scale models can continually refine their internal routing behavior to better align with task demands encountered during use. If developed responsibly, such deployment-time adaptation mechanisms could support more sustainable, efficient, and adaptable AI systems across a wide range of applications.

Acknowledgements

JG acknowledges the support of the Hector II foundation. GS acknowledges the support of the International Max Planck Research School for Intelligent Systems (IMPRS-IS). Y.Y. acknowledges support from the Alexander von Humboldt Foundation through a Humboldt Research Fellowship. Li Shen is supported by NSFC Grant (No. 62576364), Guangdong Basic and Applied Basic Research Foundation (2026B1515020071), Shenzhen Basic Research Project (Natural Science Foundation) Basic Research Key Project (NO. JCYJ20241202124430041).

References

- Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., Baker, B., Bao, H., et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. What learning algorithm is in-context learning? investigations with linear models. *arXiv preprint arXiv:2211.15661*, 2022.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dahlke, R., Klagges, H., Zecha, D., Merkel, B., Rohr, S., and Klemm, F. Mixture of tunable experts—behavior modification of deepseek-r1 at inference time. *arXiv preprint arXiv:2502.11096*, 2025.
- Dai, D., Sun, Y., Dong, L., Hao, Y., Ma, S., Sui, Z., and Wei, F. Why can gpt learn in-context? language models implicitly perform gradient descent as meta-optimizers. *arXiv preprint arXiv:2212.10559*, 2022.
- Dai, D., Deng, C., Zhao, C., Xu, R., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- Gandelsman, Y., Sun, Y., Chen, X., and Efros, A. Test-time training with masked autoencoders. *Advances in Neural Information Processing Systems*, 35:29374–29385, 2022.
- Gema, A. P., Leang, J. O. J., Hong, G., Devoto, A., Mancino, A. C. M., Saxena, R., He, X., Zhao, Y., Du, X., Madani, M. R. G., Barale, C., McHardy, R., Harris, J., Kaddour, J., van Krieken, E., and Minervini, P. Are we done with mmlu?, 2024.

- Hardt, M. and Sun, Y. Test-time training on nearest neighbors for large language models, 2024. URL <https://arxiv.org/abs/2305.18466>.
- Hübotter, J., Bongni, S., Hakimi, I., and Krause, A. Efficiently learning at test-time: Active fine-tuning of llms. *arXiv preprint arXiv:2410.08020*, 2024.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Li, X., Su, G., Sachan, M., and Geiping, J. Sample smart, not hard: Correctness-first decoding for better reasoning in llms. *arXiv preprint arXiv:2510.05987*, 2025a.
- Li, X., Zenn, J., Fadeeva, E., Su, G., Sachan, M., and Geiping, J. Efficient test-time inference via deterministic exploration of truncated decoding trees. *arXiv preprint arXiv:2604.20500*, 2026.
- Li, Z., Fan, C., and Zhou, T. Where to find grokking in llm pretraining? monitor memorization-to-generalization without test. *arXiv preprint arXiv:2506.21551*, 2025b.
- Li, Z., Li, Z., and Zhou, T. C3po: Critical-layer, core-expert, collaborative pathway optimization for test-time expert re-mixing. *arXiv preprint arXiv:2504.07964*, 2025c.
- Li, Z., Li, Z., and Zhou, T. R2-t2: Re-routing in test-time for multimodal mixture-of-experts. *arXiv preprint arXiv:2502.20395*, 2025d.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.
- Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Dengr, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Lyu, B., Murakami, S., Kamigaito, H., and Zhang, P. Routing by analogy: knn-augmented expert assignment for mixture-of-experts. *arXiv preprint arXiv:2601.02144*, 2026.
- MAA Committees. AIME problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. Olmoe: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- Osowiecki, D., Hakim, G. A. V., Noori, M., Cheraghali, M., Ben Ayed, I., and Desrosiers, C. Tttflow: Unsupervised test-time training with normalizing flow. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 2126–2134, 2023.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Shi, C., Yang, C., Zhu, X., Wang, J., Wu, T., Li, S., Cai, D., Yang, Y., and Meng, Y. Unchosen experts can contribute too: Unleashing moe models’ power by self-contrast. *Advances in Neural Information Processing Systems*, 37: 136897–136921, 2024.
- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*, pp. 9229–9248. PMLR, 2020.
- Sun, Y., Li, X., Dalal, K., Xu, J., Vikram, A., Zhang, G., Dubois, Y., Chen, X., Wang, X., Koyejo, S., et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- Team, Q. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2, 2024.
- Team, Q. Qwen3: Think deeper, act faster, 2025. URL <https://qwenlm.github.io/blog/qwen3/>. Accessed, 5(10), 2025.
- Von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, pp. 35151–35174. PMLR, 2023.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*, 2020.
- Wang, M., Chen, X., Wang, Y., He, Z., Xu, J., Liang, T., Liu, Q., Yao, Y., Wang, W., Ma, R., et al. Two experts

are all you need for steering thinking: Reinforcing cognitive effort in moe reasoning models without additional training. *arXiv preprint arXiv:2505.14681*, 2025.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Welleck, S., Bertsch, A., Finlayson, M., Schoelkopf, H., Xie, A., Neubig, G., Kulikov, I., and Harchaoui, Z. From decoding to meta-generation: Inference-time algorithms for large language models. *arXiv preprint arXiv:2406.16838*, 2024.

Xie, Y., Goyal, A., Zheng, W., Kan, M.-Y., Lillicrap, T. P., Kawaguchi, K., and Shieh, M. Monte carlo tree search boosts reasoning via iterative preference learning. *arXiv preprint arXiv:2405.00451*, 2024.

Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A. M., Le, Q. V., Laudon, J., et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35:7103–7114, 2022.

Zuo, Y., Zhang, K., Sheng, L., Qu, S., Cui, G., Zhu, X., Li, H., Zhang, Y., Long, X., Hua, E., et al. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

REPRODUCIBILITY STATEMENT

We made several efforts to ensure reproducibility. First, we provide detailed experimental settings and hyperparameters used throughout this paper in Section 4, Appendix A, and Section 5.6, and report all evaluation metrics in Section 5.1 and Section 5.2. Second, our code will be submitted with the paper, accompanied by detailed usage instructions and scripts to reproduce all reported results.

A. Experimental Settings

A.1. Benchmarks

MMLU-redux is a manually re-annotated subset of the original MMLU benchmark designed to address quality issues in the dataset. The dataset contains 3,000 questions across 30 MMLU subjects (100 questions per subject), with expert annotators identifying and categorizing various types of errors using a comprehensive error taxonomy. These errors include issues such as bad question clarity, unclear options, no correct answers, multiple correct answers, and wrong ground truth labels. MMLU-Redux provides a more reliable evaluation standard by filtering out problematic questions and offering corrected annotations where possible.

HumanEval is a benchmark dataset for evaluating code generation capabilities of large language models. The dataset consists of 164 hand-crafted programming problems, each including a function signature, docstring, body, and several unit tests (averaging 7.7 tests per problem). These challenges assess language comprehension, algorithms, and simple mathematics, with difficulty comparable to simple software interview questions.

MBPP is a code generation benchmark consisting of around 1,000 crowd-sourced Python programming problems designed to be solvable by entry-level programmers. Each problem includes a task description, code solution, and 3 automated test cases, covering programming fundamentals and standard library functionality. The dataset provides two versions: a full version with 974 problems and a sanitized version with 427 problems. The sanitized split underwent a second round of annotation to improve task descriptions, addressing issues where the original descriptions might not be sufficiently expressive to solve the tasks. This hand-verified subset provides higher-quality problem statements for more reliable evaluation of code generation models. In this paper, we use the sanitized version.

GSM8K is a dataset of 8,500 high-quality, linguistically diverse grade school math word problems created by human problem writers. The dataset is segmented into 7,473 training problems and 1,319 test problems. Each problem takes between 2 and 8 steps to solve using basic arithmetic operations, with problems designed so that a bright middle school student should be able to solve every problem. Solutions are provided in natural language format rather than pure mathematical expressions, offering insight into multi-step reasoning processes.

A.2. Baselines

C3PO We adopt C3PO (Critical-Layer, Core-Expert, Collaborative Pathway Optimization) (Li et al., 2025c) as our primary baseline method. C3PO is a test-time optimization approach designed to address the suboptimal expert pathway selection problem in Mixture of Experts (MoE) large language models. The method operates on the observation that end-to-end trained routers often produce inefficient pathways for challenging or out-of-distribution samples, leading to degraded performance on diverse downstream tasks.

The core idea of C3PO is to dynamically re-mix expert pathways during inference by leveraging successful routing patterns from a reference set. Given a reference set of m samples $\{(x_i, y_i)\}_{i=1}^m$ with their corresponding expert pathway matrices $\{\omega_i\}_{i=1}^m$ (where each $\omega_i \in \mathbb{R}^{L \times E}$ with L layers and E experts) on which the model makes correct predictions, C3PO aims to find an improved pathway matrix ω for a new test sample x that leads to more accurate outputs.

Among the three optimization strategies proposed by C3PO (gradient descent, kernel regression, and mode finding), we implement the **kernel regression** approach in our experiments. This method estimates optimal expert pathways by computing a weighted average of neighbors' pathway matrices:

$$\hat{\omega} = \frac{\sum_{i \in \mathcal{N}(x)} K(x_i, x) \omega_i}{\sum_{i \in \mathcal{N}(x)} K(x_i, x)} \quad (8)$$

where $K(\cdot, \cdot)$ is a kernel function measuring sample similarity, and $\mathcal{N}(x)$ denotes the neighborhood of x in the reference set. The final pathway is obtained through interpolation:

$$\omega \leftarrow \alpha\omega + (1 - \alpha)\hat{\omega} \quad (9)$$

where α is optimally chosen to minimize the loss function.

For our comparative evaluation, we construct reference sets for each benchmark as follows:

HumanEval and **MBPP**, we use the MBPP validation set; for **GSM8K** and **MATH500**, we utilize the GSM8K training set; and for **MMLU**, we employ the MMLU training set. From each source, we randomly sample 100 instances where the model produces the correct predictions, ensuring high-quality pathway references for the optimization process.

In-Context Learning In-Context Learning (ICL) is a fundamental capability of large language models that enables them to adapt to new tasks by using a few demonstration examples provided in the input prompt, without requiring parameter updates (Wang et al., 2022; Wei et al., 2022).

In our experimental setup, we implement ICL as a baseline across all evaluation benchmarks. The few-shot examples are carefully selected from relevant datasets to ensure domain alignment and high-quality demonstrations:

- **HumanEval** and **MBPP**: We sample 3-5 examples from the MBPP prompt collection, which provides well-crafted code generation examples with clear problem descriptions and corresponding Python solutions.
- **GSM8K**: We utilize 3-5 examples from the GSM8K training set, featuring step-by-step mathematical reasoning demonstrations that guide the model through problem-solving processes.
- **MATH500**: We sample 3-5 examples from the HYDRA-Math dataset, which offers high-quality mathematical problem-solution pairs covering various difficulty levels and mathematical domains.
- **MMLU**: We select 3-5 examples from the MMLU validation set, ensuring coverage of diverse knowledge domains while maintaining format consistency for multiple-choice questions.

For each benchmark, the few-shot examples are randomly sampled from their respective source datasets and prepended to each test query. This approach provides the model with task-specific context while maintaining consistency across different evaluation scenarios. The number of examples (3-5) is chosen to balance between providing sufficient context and avoiding excessive prompt length that might degrade model performance.

A.3. Model Selection

OLMoE is a fully open-source MoE language model developed by the Allen Institute for AI and Contextual AI (Muennighoff et al., 2024). The model employs a decoder-only transformer architecture with 1 billion active and 7 billion total parameters. Each MoE layer contains 64 experts, of which 8 are activated per input token through a learned router network. This sparse activation mechanism enables computational efficiency similar to dense 1B parameter models while leveraging the full 7B parameter capacity.

In our experiments, we utilize OLMoE-1B-7B as the base model to evaluate expert pathway optimization strategies across diverse benchmarks.

DeepSeek-V2-Lite is a smaller variant of the DeepSeek-V2 model family, developed by DeepSeek-AI (Liu et al., 2024). The model employs innovative architectures, including Multi-head Latent Attention (MLA) and DeepSeekMoE, with 15.7 billion total parameters and 2.4 billion activated parameters per token. DeepSeek-V2-Lite has 27 layers with a hidden dimension of 2048, utilizing MLA with 16 attention heads, where each head has a dimension of 128.

The model adopts the DeepSeekMoE architecture, where all feed-forward networks except the first layer are replaced with MoE layers. Each MoE layer consists of 2 shared experts and 64 routed experts, with 6 experts activated for each token. This sparse activation mechanism enables computational efficiency while maintaining strong performance across diverse tasks.

In our experiments, we use DeepSeek-V2-Lite to evaluate expert pathway optimization strategies, leveraging its balance between model capacity and computational efficiency for comprehensive benchmark evaluation.

Qwen1.5-MoE is developed by the Qwen team (Team, 2024). The model is upcycled from the dense Qwen-1.8B model, featuring 14.3 billion total parameters with 2.7 billion activated parameters during runtime. Despite using only 2.7B active parameters, the model achieves comparable performance to Qwen1.5-7B while requiring 75% fewer training resources and demonstrating 1.74x faster inference speed.

The model employs a fine-grained expert architecture with 64 experts total, consisting of 4 shared experts that are always activated and 60 routing experts with 4 activated per token. This configuration represents an 8-fold increase in expert count compared to conventional MoE setups, enabling higher model capacity without proportional parameter increases. The fine-grained expert design partitions a single FFN into multiple segments, each serving as an individual expert, allowing for more specialized knowledge representation.

In our experiments, we utilize Qwen1.5-MoE to evaluate expert pathway optimization strategies, leveraging its efficient architecture that balances performance and computational cost across diverse evaluation benchmarks.

B. Method Details

B.1. Test-Time MoE Rerouting

We elaborate the rerouting algorithmic pipeline in Algorithm 1, which details the step-by-step procedure for adaptive expert selection and pathway calibration at test time.

B.2. Pathway Differences

We first examine how expert pathways change after rerouting using edit distance following (Li et al., 2025b). For input x_i , we define its pathway s_i as the ordered sequence of selected experts across L MoE layers as $s_i = \text{concat}(e_1^{(i)}, e_2^{(i)}, \dots, e_L^{(i)})$, where $e_\ell^{(i)}$ represents expert indices at layer ℓ as comma-separated strings (e.g., ‘3,1,5’), joined across layers with hyphens. We quantify pathway differences using Levenshtein edit distance as $D_{\text{path}}(s_i, s_j) = \text{EditDistance}(s_i, s_j)$. This captures mismatches in expert selection, and pathway shifts.

B.3. Comparison With Shannon Entropy

Consider a discrete probability distribution $\mathbf{p} = (p_1, p_2, \dots, p_n)$ over n possible outcomes, where $\sum_{i=1}^n p_i = 1$ and $p_i > 0$.

Shannon entropy:

$$H_{\text{Shannon}}(\mathbf{p}) = - \sum_{i=1}^n p_i \log p_i \quad (10)$$

Our specialization measure:

$$H_{\text{specialization}}(\mathbf{p}) = - \sum_{i=1}^n \log p_i \quad (11)$$

The key distinction is that Shannon entropy weights each term by p_i , while our measure treats all outcomes equally. This leads to opposite monotonicity with respect to distribution concentration:

- **Shannon entropy:** maximized by uniform distributions, minimized by one-hot distributions (measures uncertainty).
- **Our specialization measure:** minimized by uniform distributions, grows unboundedly toward one-hot distributions (measures confidence/peakedness).

This inverse relationship makes our measure suitable for selecting high specialization score routing layers, as we prioritize sharp, decisive expert selections over uncertain routing patterns.

C. Additional Analysis

C.1. Layer-wise confidence distributions across different tasks

We visualize the confidence distributions across different tasks using DeepSeek-V2-lite as an example. As illustrated in Figure 5, activation patterns across experts and layers are highly task-specific. For instance, math tasks (Math500) and code tasks (HumanEval) exhibit distinct confidence patterns across layers, with math tasks showing higher confidence in middle layers (layers 7-14) while code tasks demonstrate more distributed confidence patterns with peaks in later layers (layers 15-18). Notably, both tasks show generally higher confidence concentrated in the middle-to-later layers compared to early layers.

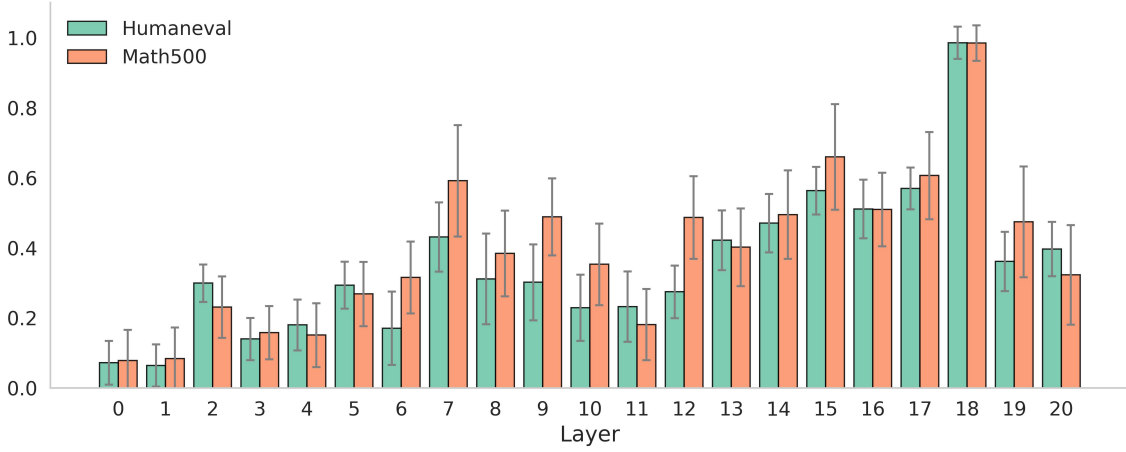


Figure 5. Layer-wise confidence distributions across different tasks in DeepSeek-V2-lite-MoE

C.2. Sensitivity Analysis of Hyperparameters

Analysis of Optimization Interval. As shown in Figure 6, we investigate the impact of optimization interval m (measured in tokens) on performance across five benchmarks. Our test-time rerouting method consistently outperforms the baseline across all interval settings, demonstrating robustness to this hyperparameter. The results reveal that intervals of 128-160 tokens achieve optimal performance, effectively balancing routing adaptability with computational efficiency.

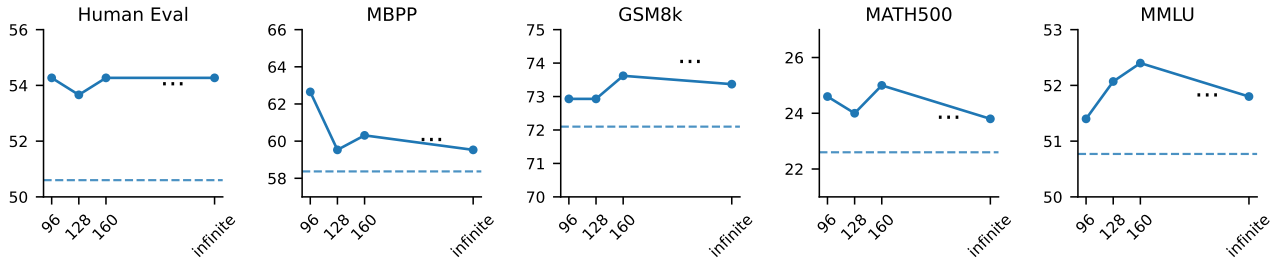


Figure 6. Effect of optimization interval on performance across five benchmarks. The dashed line represents baseline results without rerouting. Our method consistently outperforms the baseline.

Analysis of Optimization Steps. We conduct experiments using the DeepSeek V2 model, evaluating its performance on the HumanEval across different optimization step settings. Table 6 presents the performance sensitivity to the number of optimization steps T during test-time adaptation. The results show that performance peaks at $T = 4$ (54.88%), with $T = 3$ -5 forming an optimal range (53.83%-54.88%). This sweet spot allows sufficient adaptation without over-fitting to individual test instances.

Beyond $T = 5$, performance gradually declines, suggesting over-adaptation where the model begins to fit noise rather than meaningful patterns in the test data. Interestingly, even with suboptimal step counts, our method maintains consistent

improvements across $T = 3-7$ (all $> 52\%$), demonstrating robustness to this hyperparameter. The relatively flat performance curve within the optimal range indicates that practitioners can select values in this region without significant performance penalties, making the method practical for deployment.

Table 6. Performance (%) across different optimization steps T on HumanEval benchmark. The optimal range is $T = 3-5$, with peak performance at $T = 4$.

T	1	2	3	4	5
Performance	50.60	50.17	53.83	54.88	54.27
T	6	7	8	9	10
Performance	52.95	53.83	51.56	51.83	51.83

C.3. Impact on Sample Diversity in Parallel Generation.

Table 7. Diversity evaluation results comparison

Metric	Baseline	Ours
Cosine Div.	0.390 ± 0.157	0.380 ± 0.150
Semantic Div.	0.012 ± 0.007	0.012 ± 0.008

Impact on Sample Diversity in Parallel Generation. While our online adaptation method improves routing efficiency, we investigate whether the adapted routing decisions might reduce sample diversity when generating multiple sequences in parallel. To quantify this effect, we evaluate the diversity of generated samples using two complementary metrics: semantic diversity based on CodeBERT embeddings (Feng et al., 2020), which captures functional similarities between code snippets, and TF-IDF-based cosine diversity, which measures lexical variation. We used the DeepSeek MoE model with default settings to generate 10 samples on the HumanEval task. Table 7 shows that our method maintains comparable diversity to the baseline across both metrics. Cosine diversity scores are nearly identical (0.380 ± 0.150 vs. 0.390 ± 0.157), as are semantic diversity scores (0.012 ± 0.008 vs. 0.012 ± 0.007). These results demonstrate that our online adaptation preserves sample diversity while improving routing efficiency.

C.4. Impact on Internal Confidence in Generation

We further investigate how our method influences the internal confidence during generation. The central hypothesis is that model uncertainty, quantified using information-theoretic measures such as entropy or softmax margins, can serve as a reliable indicator of generation reliability.

Let $P(y_t|y_{<t}, x)$ denote the output distribution of the model at time step t , conditioned on previously decoded tokens $y_{<t}$ and the input sequence x . For each position t , a scalar confidence score C_t is computed that reflects the model’s certainty on its prediction.

Here, we use two kinds of metrics to estimate the confidence score: (1) entropy-based confidence and (2) softmax margin confidence.

Entropy-based Confidence: Entropy serves as a fundamental measure of uncertainty in probabilistic models. For token-level generating, the entropy of the model’s predictive distribution is given by: $C_t^{(ent)} = -\sum_{y \in V} P(y|y_{<t}, x) \times \log P(y|y_{<t}, x)$. Here, V denotes the output vocabulary, and $P(\cdot|y_{<t}, x)$ is the model’s probability distribution over the next token given the previous context and input. A lower value of $C_t^{(ent)}$ corresponds to a sharper (more peaked) distribution, implying higher confidence in the predicted token. Conversely, a higher entropy indicates greater uncertainty and suggests a more conservative generation.

Softmax margin confidence: Softmax margin confidence captures the model’s certainty by evaluating the probability gap between the top two predicted tokens after applying the softmax function. At each decoding step t , let $y^{(1)}$ and $y^{(2)}$ denote the top-1 and top-2 predicted tokens by the model, and let their softmax probabilities be: $P^{(1)} = P(y^{(1)}|y_{<t}, x)$ and $P^{(2)} = P(y^{(2)}|y_{<t}, x)$. The softmax margin confidence is then computed as: $C_t^{(soft)} = P^{(1)} - P^{(2)}$.

The softmax margin confidence reflects how decisively the model prefers its most likely token over the next best alternative,

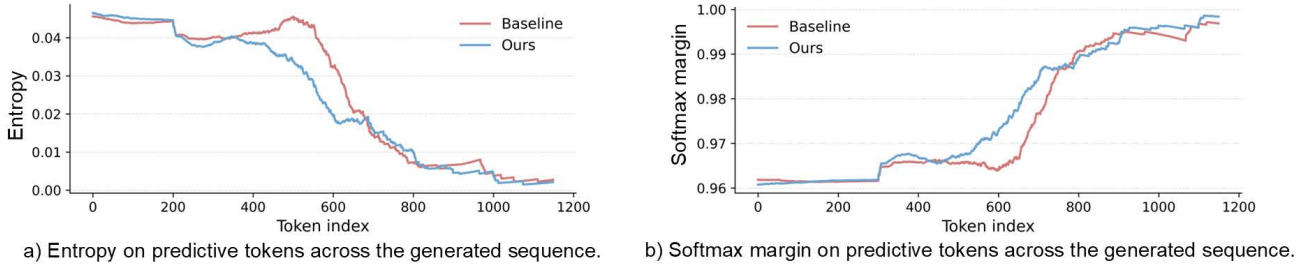


Figure 7. Confidence metrics across token positions during sequence generation.

judged by the normalized probability distribution. A higher value of $C_t^{(soft)}$ implies strong confidence in the top choice, whereas smaller values indicate ambiguity, with similar likelihoods assigned to multiple candidates.

As shown in Figure 7, compared to the baseline, our method demonstrates notable improvements in confidence characteristics across the generated sequence. In Figure 7(a), the entropy-based confidence reveals that our approach achieves lower entropy values, particularly in the middle and later stages of generation (token index 400-1000), indicating more peaked probability distributions and higher prediction certainty. This suggests that our method produces more confident and decisive token predictions. Meanwhile, Figure 7(b) shows that our method consistently maintains higher softmax margin values throughout the generation process, especially after token index 400, demonstrating a clearer preference for the top predicted token over alternatives. The increased margin between top-1 and top-2 predictions indicates that our method reduces ambiguity and enhances the model’s decision confidence. These results collectively demonstrate that our approach enhances the model’s internal confidence, leading to more reliable and deterministic outputs.

C.5. What if We Optimize the Full Linear Routing Layer?

To validate our design choice of bias-only optimization, we compare it against alternative parameter adaptation strategies. Table 8 presents the results across different optimization approaches on the HumanEval benchmark. Results show that: (1) Full linear layer optimization leads to complete performance collapse, indicating severe instability when optimizing all routing parameters at test time. (2) Low-rank LoRA (rank-1), despite lower capacity than full layer optimization, underperforms even the baseline, demonstrating that **increased capacity does not guarantee better test-time adaptation**. Our bias-only approach achieves the best performance (54.26%) with minimal parameters, effectively balancing adaptation capability and optimization stability.

Table 8. Performance comparison of different parameter adaptation strategies on HumanEval benchmark.

Method	HumanEval (%)
Baseline	50.60
Ours (bias-only)	54.26
Rank-1 LoRA	46.80
Full linear layer	collapse

C.6. Performance Analysis Across Different Sequence Lengths

We conducted additional analysis on HumanEval by stratifying samples based on their sequence lengths and computing accuracy for each bucket. As shown in Table 11, our method demonstrates increasing performance gains as sequence length grows. For short sequences (< 256 tokens), both methods achieve perfect accuracy (100.0%). However, as sequences become longer, our method shows progressively larger improvements: +2.9% for 256-384 tokens, +3.6% for 384-512 tokens, and +2.4% for 512-640 tokens. Most notably, for very long sequences exceeding 640 tokens, our method achieves a substantial +13.3% improvement over the baseline (23.3% vs. 10.0%). This trend validates that test-time routing optimization becomes increasingly beneficial as context length grows.

Table 9. Memory usage for generation and refinement phases across generated lengths.

Length	Generation (GiB)	Refinement (GiB)
0	29.44	30.97
128	29.56	31.54
256	29.67	32.41
384	29.80	33.46
512	29.91	34.32
640	30.00	34.92
768	30.05	35.99

C.7. Memory overhead.

The refinement stage introduces only a modest memory overhead over standard inference. This is because our method does not duplicate the base model or perform full-model fine-tuning. During refinement, the pretrained model weights are already resident in GPU memory, as in ordinary generation, and the only trainable variables are the compact δ parameters. For example, in DeepSeek-V2-Lite, δ contains only 1,792 floating-point values. Therefore, the additional memory cost is not dominated by model parameters, gradients, or optimizer states, but mainly by the cached forward activations required for the backward pass through the refinement objective. Empirically, this leads to substantially lower memory consumption than full-model backpropagation. As shown in Table 9, the generation-only memory ranges from 29.44 to 30.05 GiB as the generated length increases from 0 to 768 tokens, whereas the refinement phase ranges from 30.97 to 35.99 GiB. The observed overhead is therefore far below a naive $2\times$ increase over inference memory, confirming that the proposed refinement is memory-efficient in practice.

Table 10. Performance comparison under pass@ k and majority voting metrics.

Method	pass@2	pass@4	pass@8	Maj@2	Maj@4	Maj@8	Avg
Baseline	0.84	0.88	0.90	0.71	0.76	0.80	0.815
Ours	0.88	0.92	0.93	0.74	0.79	0.83	0.848

C.8. Evaluation on ultra-large models.

We additionally evaluated our method on GPT-OSS-120B using AIME24. Our method improves the average score from 0.815 to 0.848 and outperforms the baseline across all six metrics, including pass@2, pass@4, pass@8, Maj@2, Maj@4, and Maj@8. This result suggests that expert gating remains a meaningful optimization target even in ultra-large MoE models, where lightweight gating refinement can still yield consistent performance gains.

C.9. Cases Study

The refinement stage introduces only a modest memory overhead over standard inference. This is because our method does not duplicate the base model or perform full-model fine-tuning. During refinement, the pretrained model weights are already resident in GPU memory, as in ordinary generation, and the only trainable variables are the compact δ parameters. For example, in DeepSeek-V2-Lite, δ contains only 1,792 floating-point values. Therefore, the additional memory cost is not dominated by model parameters, gradients, or optimizer states, but mainly by the cached forward activations required for the backward pass through the refinement objective. Empirically, this leads to substantially lower memory consumption than full-model backpropagation. As shown in Table 9, the generation-only memory ranges from 29.44 to 30.05 GiB as the generated length increases from 0 to 768 tokens, whereas the refinement phase ranges from 30.97 to 35.99 GiB. The observed overhead is therefore far below a naive $2\times$ increase over inference memory, confirming that the proposed refinement is memory-efficient in practice.

Algorithm 1 Test-Time MoE Rerouting

Input: Pre-trained MoE model $\mathcal{M}$; input prompt $\mathbf{x} = (x_1, \dots, x_n)$; optimization steps n ; generation interval m ; optimizer $\mathcal{O}$.

Output: Generated response $\mathbf{y}$.

// Parameter Initialization

Initialize $\delta^{(l)} = \mathbf{0} \in \mathbb{R}^N$ for all layers $l \in \mathcal{L}$;

// Phase 1: In-Context Routing Optimization

$\mathbf{x}_{\text{current}} = \mathbf{x}, T = |\mathbf{x}|$;

Select layers $\mathcal{S} = \text{TopK}(\{C^{(l)}\}_{l=1}^L, r)$ or compute soft weights $\{w_l\}_{l=1}^L$;

for $i = 1$ to n **do**

 Compute loss $\mathcal{L}(\{\delta^{(l)}\}_{l=1}^L) = -\sum_{j=1}^{T-1} \log p(x_{j+1} \mid x_{1:j}, \{\delta^{(l)}\}_{l=1}^L)$;

if hard selection **then**

for $l \in \mathcal{S}$ **do**

$\delta^{(l)} \leftarrow \mathcal{O}(\delta^{(l)}, \nabla_{\delta^{(l)}} \mathcal{L})$;

end for

else // soft weighting

for $l = 1$ to L **do**

$\delta^{(l)} \leftarrow \mathcal{O}(\delta^{(l)}, w_l \nabla_{\delta^{(l)}} \mathcal{L})$;

end for

end if

end for

// Phase 2: Steered Generation with Periodic Re-optimization

Initialize $\mathbf{y} = ()$;

repeat

// Generate m tokens using optimized routing parameters

for $k = 1$ to m **do**

 Generate $x_{\text{next}} \sim p(\cdot \mid \mathbf{x}_{\text{current}}, \{\delta^{(l)}\}_{l=1}^L)$;

 Append x_{next} to $\mathbf{y}$ and $\mathbf{x}_{\text{current}}$;

end for

// Re-optimize with extended context

$T = |\mathbf{x}_{\text{current}}|$;

 Select layers $\mathcal{S} = \text{TopK}(\{C^{(l)}\}_{l=1}^L, r)$ or compute soft weights $\{w_l\}_{l=1}^L$;

for $i = 1$ to n **do**

 Compute loss $\mathcal{L}(\{\delta^{(l)}\}_{l=1}^L) = -\sum_{j=1}^{T-1} \log p(x_{\text{current},j+1} \mid x_{\text{current},1:j}, \{\delta^{(l)}\}_{l=1}^L)$;

if hard selection **then**

for $l \in \mathcal{S}$ **do**

$\delta^{(l)} \leftarrow \mathcal{O}(\delta^{(l)}, \nabla_{\delta^{(l)}} \mathcal{L})$;

end for

else // soft weighting

for $l = 1$ to L **do**

$\delta^{(l)} \leftarrow \mathcal{O}(\delta^{(l)}, w_l \nabla_{\delta^{(l)}} \mathcal{L})$;

end for

end if

end for

until end-of-sequence or max length reached;

return $\mathbf{y}$;

Table 11. Performance comparison across different sequence lengths on HumanEval. Our method shows increasing gains on longer sequences.

Length Range	Ours	Baseline	Gain
< 256	100.0%	100.0%	+0.0%
256–384	73.5%	70.6%	+2.9%
384–512	61.8%	58.2%	+3.6%
512–640	46.3%	43.9%	+2.4%
> 640	23.3%	10.0%	+13.3%

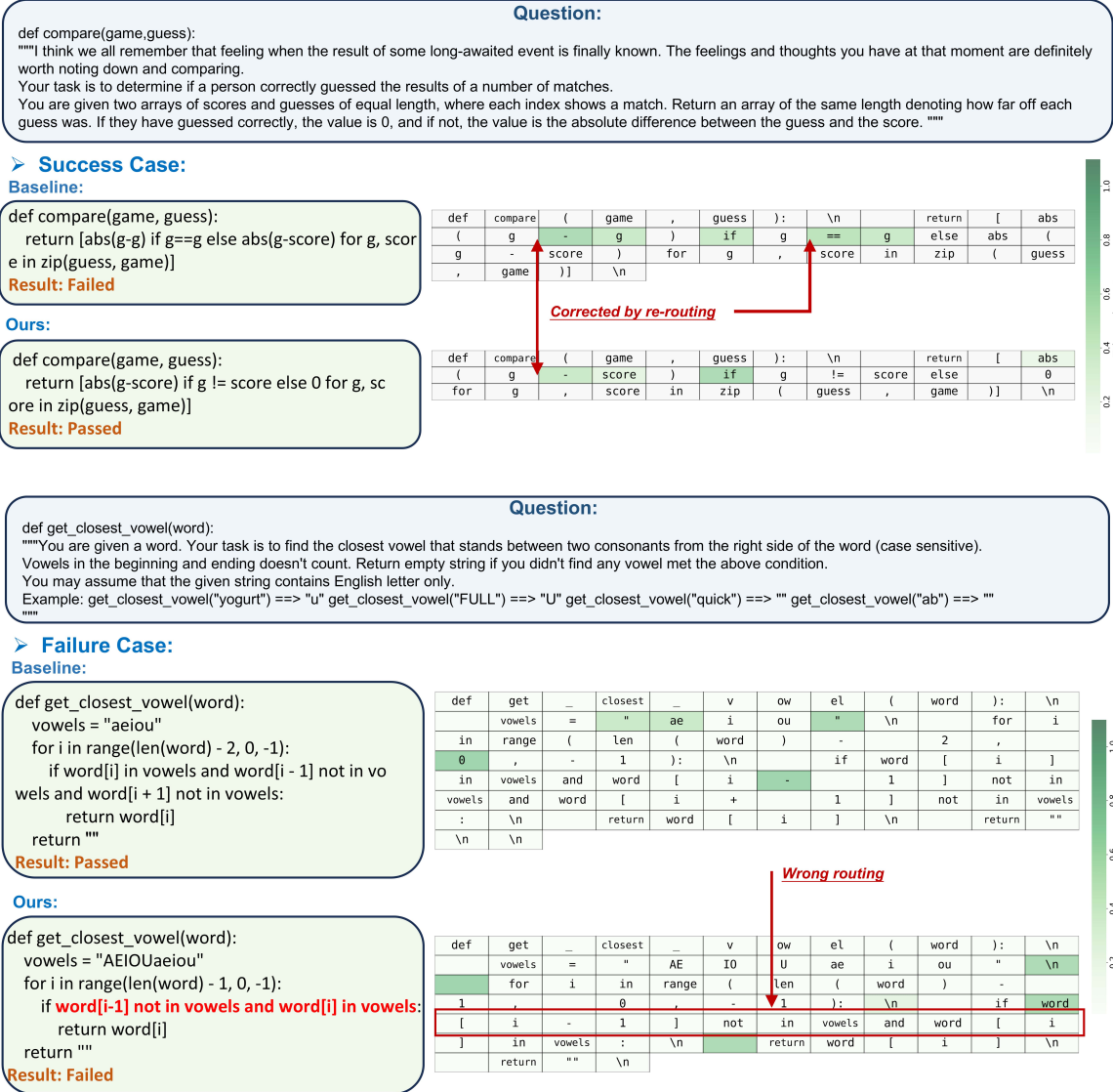


Figure 8. Representative rewiring case studies (top: one success case/ bottom: one failure case).